UNIVERZITET U NIŠU
ELEKTRONSKI
FAKULTET

Fizičko projektovanje baze i optimizacija podataka u MS SQL bazi podataka

Seminarski rad

Studijski program: Računarstvo i informatika
Modul: Softversko inženjerstvo

Student:
Anastasija Savov
Broj indeksa: 1924

Mentor:
Prof. dr Aleksandar Stanimirović

Niš, 2025.

Sadržaj

1 Uvod **3**

2 Fizička organizacija podataka **4**
2.1 Organizacija podataka na disku: stranice i extent-i 4
2.2 Row-store vs Column-store skladištenje 5
2.2.1 Row-store (skladištenje po redovima) 5
2.2.2 Column-store (skladištenje po kolonama) 6
2.3 Heap tabele vs klasterovane tabele 7
2.3.1 Heap tabele 7
2.3.2 Klasterovane tabele 8
2.3.3 Kada koristiti heap, a kada klasterovanu tabelu? 9
2.4 Kompresija podataka 9
2.4.1 ROW kompresija 9
2.4.2 PAGE kompresija 9
2.4.3 Gde se kompresija fizički primenjuje 9
2.4.4 Uticaj kompresije na performanse 11
2.5 Uticaj organizacije podataka na OLTP vs OLAP sisteme 11
2.5.1 OLTP (Online Transaction Processing) 12
2.5.2 OLAP (Online Analytical Processing) 12

3 Radno opterećenje (Query Workload) **14**
3.1 Analiza workload-a po koracima 14
3.1.1 Najvažniji upiti i koliko se često koriste 14
3.1.2 Najvažnija ažuriranja i koliko se često izvršavaju 14
3.1.3 Željene performanse za upite i ažuriranja 15
3.2 Preporuke za odluke nakon analize workload-a 15
3.3 Tuning upita (Query tuning) 15
3.3.1 SARGable uslovi u WHERE klauzuli 15
3.3.2 Smanjenje čitanja 16
3.3.3 JOIN tuning i rano filtriranje 16
3.3.4 Poređenje pristupa: JOIN vs CTE vs CROSS APPLY 16

4 Projektovanje tabela i relacija **18**
4.1 Izbor tipova podataka 18
4.2 Projektovanje primarnog ključa 19
4.3 Strani ključevi i referencijalni integritet 19
4.4 Ograničenja (Constraints) 20
4.5 Particionisanje tabela 21
4.5.1 HASH particionisanje 21
4.5.2 LIST particionisanje 22
4.5.3 Poredjenje strategija particionisanja 23
4.6 Filegroup-e i organizacija skladišta 24
4.7 Preporuke za projektovanje tabela 25
4.8 Preporuke za podešavanje konceptualne sheme 25

5 Projektovanje indeksa 27
5.1 B-tree struktura . . . 27
5.2 Klasterovani indeksi . . . 27
5.2.1 Izbor ključa klasterovanog indeksa . . . 27
5.3 Neklasterizovani indeksi . . . 28
5.3.1 Covering indeksi i INCLUDE kolone . . . 28
5.3.2 Kompozitni indeksi . . . 28
5.3.3 Filtrirani indeksi . . . 29
5.4 Columnstore indeksi . . . 29
5.5 Opšta uputstva za kreiranje indeksa . . . 30

6 Ko-klasterovanje (Co-clustering) 31
6.1 Princip ko-klasterovanja . . . 31
6.2 Ko-klasterovanje u MS SQL Server-u . . . 31
6.3 Preporuke za primenu ko-klasterovanja . . . 32

7 Zaključak 33

8 Literatura 34

1 Uvod

Fizičko projektovanje baze podataka predstavlja ključnu fazu u razvoju informacionih sistema koja direktno utiče na performanse, skalabilnost i pouzdanost aplikacija. Za razliku od logičkog projektovanja, koje se bavi konceptualnom organizacijom podataka i relacijama između entiteta, fizičko projektovanje se fokusira na način na koji se podaci stvarno skladište na disku, kako im se pristupa i kako se optimizuju za konkretno radno opterećenje [1].

Microsoft SQL Server, kao jedan od najpopularnijih sistema za upravljanje relacionim bazama podataka, pruža bogat skup mehanizama za fizičko projektovanje i podešavanje (tuning) baze podataka. Od analize radnog opterećenja i izbora indeksa, preko podešavanja upita i pogleda, do upravljanja konkurentnošću i merenjem performansi putem testova performansi - svaka odluka u fizičkom projektovanju ima merljiv uticaj na performanse sistema [2].

Ovaj seminarski rad ima za cilj da pruži pregled ključnih aspekata fizičkog projektovanja i podešavanja baze podataka u MS SQL Server-u. Rad obuhvata analizu radnog opterećenja, projektovanje tabela i relacija sa preporukama za izbor tipova podataka, ključeva i ograničenja, strategije projektovanja indeksa, ko-klasterovanje i optimizaciju upita.

2 Fizička organizacija podataka

Razumevanje načina na koji SQL Server fizički skladište podatke je osnova za donošenje ispravnih odluka o fizičkom projektovanju. Ovaj odeljak objašnjava internu arhitekturu skladištenja i njene implikacije na performanse [2].

2.1 Organizacija podataka na disku: stranice i extent-i

SQL Server organizuje podatke u hijerarhijskoj strukturi koja se sastoji od stranica (pages) i extent-a [2]:

- **Stranica (Page)** - Osnovna jedinica skladištenja i I/O operacija. Svaka stranica ima fiksnu veličinu od **8 KB** (8192 bajta). Od toga, 96 bajtova zauzima zaglavlje stranice, a preostali prostor je dostupan za podatke i meta-informacije o redovima.
- **Extent** - Grupa od **8 uzastopnih stranica** (64 KB). SQL Server alocira prostor na nivou extent-a radi efikasnosti. Postoje dve vrste extent-a:
  - **Uniform extent** - Sve stranice pripadaju jednom objektu (tabeli ili indeksu)
  - **Mixed extent** - Stranice mogu pripadati različitim objektima (koristi se za male tabele)

Page 0	Page 1	Page 2	Page 3	Page 4	Page 5	Page 6	Page 7
1 Extent = 8 Pages = 64 KB							

Slika 1: Struktura extent-a u SQL Server-u

SQL Server podržava nekoliko tipova stranica:

- **Data pages** - Skladište redove podataka (osim LOB tipova)
- **Index pages** - Skladište indeksne zapise
- **LOB pages** - Skladište velike objekte (text, ntext, image, varchar(max), varbinary(max))
- **GAM/SGAM pages** - Prate alokaciju extent-a
- **PFS pages** - Prate slobodan prostor na stranicama
- **IAM stranice (Index Allocation Map)** - Posebne stranice koje prate koje extent-e koristi svaki objekat u bazi podataka

```sql
-- Broj stranica i extent-a po tabeli
SELECT
    t.name AS TableName,
    p.rows AS RowCount,
    SUM(a.total_pages) AS TotalPages,
    SUM(a.total_pages) * 8 / 1024 AS TotalSpaceMB,
    SUM(a.used_pages) AS UsedPages,
    SUM(a.data_pages) AS DataPages
FROM sys.tables t
```

```sql
INNER JOIN sys.indexes i ON t.object_id = i.object_id
INNER JOIN sys.partitions p ON i.object_id = p.object_id
    AND i.index_id = p.index_id
INNER JOIN sys.allocation_units a ON p.partition_id = a.container_id
GROUP BY t.name, p.rows
ORDER BY TotalPages DESC;

-- Detaljna analiza stranice (nedokumentovana komanda)
DBCC PAGE ('TestDB', 1, 328, 3) WITH TABLERESULTS;
```

Listing 1: Pregled informacija o stranicama i extent-ima

Results | Messages

	TableName	RowCount	TotalPages	TotalSpaceMB	UsedPages	DataPages
1	LogPoruke	100000	100665	786	100646	100645
2	Arhiva	10000	2182	17	2060	2034
3	Prodaja_RowStore	100010	489	3	483	481
4	Prodaja_ColumnStore	100010	9	0	2	0

Slika 2: Prikaz stranica i zauzetost prostora u SQL Server-u

2.2 Row-store vs Column-store skladištenje

Postoje dva različita pristupa organizaciji podataka na disku: **skladištenje po redovima (row-store)** i **skladištenje po kolonama (column-store)** [2, 3].

2.2.1 Row-store (skladištenje po redovima)

U row-store modelu, svi atributi jednog reda se skladište zajedno na istoj stranici. Ovo je tradicionalni model koji SQL Server koristi za heap tabele i B-tree indekse.

```
Stranica 1: [Red1: ID,Ime,Prezime,Grad] [Red2: ID,Ime,Prezime,Grad] ...
Stranica 2: [Red3: ID,Ime,Prezime,Grad] [Red4: ID,Ime,Prezime,Grad] ...
```

Prednosti row-store modela:

- Efikasan za OLTP operacije - pristup celom redu zahteva čitanje samo jedne stranice
- Brze INSERT, UPDATE i DELETE operacije
- Efikasan za upite koji vraćaju sve kolone malog broja redova

Nedostaci row-store modela:

- Neefikasan za analitičke upite koji čitaju samo nekoliko kolona iz miliona redova
- Ograničene mogućnosti kompresije (različiti tipovi podataka u istom bloku)

2.2.2 Column-store (skladištenje po kolonama)

U column-store modelu, vrednosti iste kolone se skladište zajedno. SQL Server implementira ovo kroz Columnstore indekse (dostupne od verzije 2012).

```
Segment ID:      [1, 2, 3, 4, 5, ...]
Segment Ime:     [Ana, Marko, Jovan, Milica, ...]
Segment Prezime: [Peric, Jovic, Stanic, Nikolic, ...]
Segment Grad:    [Nis, Beograd, Nis, Novi Sad, ...]
```

Prednosti column-store modela:

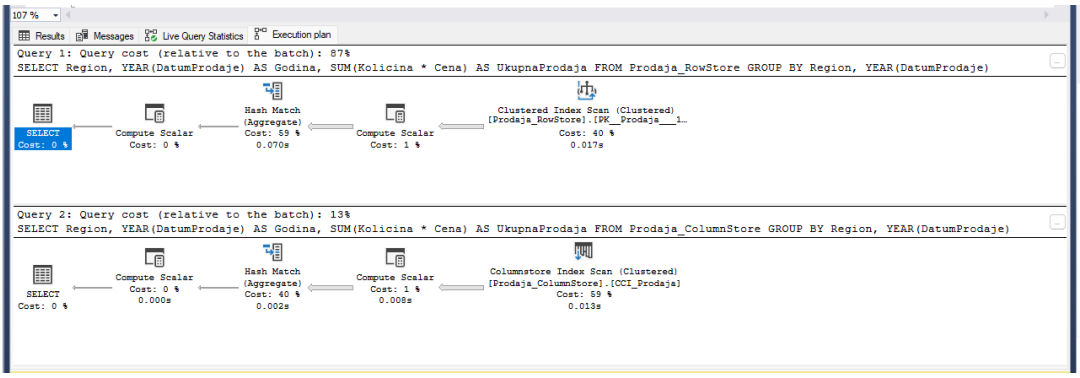
- Izuzetna kompresija - iste vrednosti u koloni se lako kompresuju (često 10x manja veličina)
- Efikasno za analitičke upite - čitaju se samo potrebne kolone
- Batch processing - operacije se izvršavaju nad grupama redova, a ne pojedinačno
- Vektorizovano izvršavanje - iskorišćava moderne CPU instrukcije (SIMD)

Nedostaci column-store modela:

- Sporije pojedinačne UPDATE operacije
- Overhead pri rekonstrukciji kompletnog reda
- Delta store overhead za česte INSERT operacije

```sql
-- Row-store tabela (tradicionalna)
CREATE TABLE Prodaja_RowStore (
    ProdajaID INT IDENTITY PRIMARY KEY,
    DatumProdaje DATE,
    ProizvodID INT,
    Kolicina INT,
    Cena DECIMAL(10,2),
    Region VARCHAR(50)
);

-- Column-store tabela (za analitiku)
CREATE TABLE Prodaja_ColumnStore (
    ProdajaID INT,
    DatumProdaje DATE,
    ProizvodID INT,
    Kolicina INT,
    Cena DECIMAL(10,2),
    Region VARCHAR(50),
    INDEX CCI_Prodaja CLUSTERED COLUMNSTORE
);

-- Analiticki upit - column-store je znacajno brzi
SELECT
    Region,
    YEAR(DatumProdaje) AS Godina,
    SUM(Kolicina * Cena) AS UkupnaProdaja
FROM Prodaja_ColumnStore
GROUP BY Region, YEAR(DatumProdaje);
```

Listing 2: Poređenje row-store i column-store pristupa



Slika 3: Poređenje rezultata za row-store i column-store pristup

2.3 Heap tabele vs klasterovane tabele

U SQL Server-u, tabela može biti organizovana kao **heap** (bez klasterovanog indeksa) ili kao **klasterovana tabela** (sa klasterovanim indeksom) [2].

2.3.1 Heap tabele

Heap je tabela bez klasterovanog indeksa. Redovi se skladište bez određenog redosleda, a lokacija svakog reda se identifikuje putem RID (Row Identifier) koji se sastoji od FileID:PageID:SlotID.

Karakteristike heap tabela:

- Redovi se dodaju na prvu dostupnu lokaciju (bez sortiranja)
- Forwarding pointers - kada se red poveća i ne stane na originalnu stranicu, kreira se pokazivač na novu lokaciju
- Neklasterovani indeksi koriste RID kao pokazivač
- Table Scan je jedini način za pretragu bez indeksa

```sql
-- Kreiranje heap tabele (bez PRIMARY KEY CLUSTERED)
CREATE TABLE LogPoruke (
    LogID INT IDENTITY,
    Poruka VARCHAR(MAX),
    DatumVreme DATETIME2 DEFAULT SYSDATETIME()
);

-- Provera forwarding pointer-a
SELECT
    OBJECT_NAME(object_id) AS TableName,
    forwarded_record_count,
    page_count
FROM sys.dm_db_index_physical_stats(
    DB_ID(), OBJECT_ID('LogPoruke'), 0, NULL, 'DETAILED')
WHERE index_id = 0;  -- 0 = heap
```

Listing 3: Heap tabela i problemi sa forwarding pointers

Results | Messages

	TableName	forwarded_record_count	page_count
1	LogPoruke	99999	100645

Slika 4: Prikaz forwarding pointer-a za tabelu LogPoruke

2.3.2 Klasterovane tabele

Klasterovana tabela ima klasterovani indeks koji definiše fizički redosled redova. Leaf nivo klasterovanog indeksa **sadrži same podatke**.

Karakteristike klasterovanih tabela:

- Redovi su fizički sortirani po ključu klasterovanog indeksa
- Nema forwarding pointer problema
- Neklasterovani indeksi koriste klasterovani ključ kao pokazivač
- Range scan je veoma efikasan

```sql
-- Klasterovana tabela
CREATE TABLE Narudzbine_Clustered (
    NarudzbinaID INT IDENTITY,
    DatumNarudzbine DATE,
    KupacID INT,
    CONSTRAINT PK_Narudzbine_Clustered
        PRIMARY KEY CLUSTERED (NarudzbinaID)
);

-- Heap tabela (eksplicitno bez klasterovanog indeksa)
CREATE TABLE Narudzbine_Heap (
    NarudzbinaID INT IDENTITY,
    DatumNarudzbine DATE,
    KupacID INT,
    CONSTRAINT PK_Narudzbine_Heap
        PRIMARY KEY NONCLUSTERED (NarudzbinaID)
);
```

Listing 4: Poređenje heap i klasterovane tabele

2.3.3 Kada koristiti heap, a kada klasterovanu tabelu?

Tabela 1: Poređenje heap i klasterovanih tabela

Aspekt	**Heap**	**Klasterovana tabela**
INSERT performanse	Brže (bez održavanja redosleda)	Sporije ako ključ nije monoton
Range upiti	Neefikasni (Table Scan)	Veoma efikasni (Index Seek + Scan)
Fragmentacija	Forwarding pointers	Page splits
Prostor	Manji (bez indeksne strukture)	Veći (B-tree overhead)
Preporuka	Staging tabele, log tabele	Većina OLTP tabela

2.4 Kompresija podataka

SQL Server podržava kompresiju podataka na nivou reda (ROW) i stranice (PAGE) za row-store tabele i indekse, što može smanjiti prostor na disku i broj I/O operacija [4].

2.4.1 ROW kompresija

ROW kompresija optimizuje skladištenje vrednosti za pojedinačne redove:

- Fiksni tipovi podataka se čuvaju u internom formatu promenljive dužine
- Tipično smanjuje veličinu redova sa mnogo fiksnih kolona i/ili kratkih vrednosti
- NULL i neke numeričke vrednosti (npr. 0) mogu zauzimati minimalno ili 0 bajta u delu za podatke
- Ne koristi međurednu kompresiju (svaki red se kompresuje nezavisno)

2.4.2 PAGE kompresija

PAGE kompresija uključuje ROW kompresiju i dodatne tehnike na nivou cele stranice:

- **Prefix compression** - zajednički prefiksi se izdvajaju i referenciraju
- **Dictionary compression** - ponavljajući obrasci na stranici se zamenjuju kraćim referencama
- Obično daje veću uštedu prostora od ROW kompresije, ali sa većim CPU overheadom

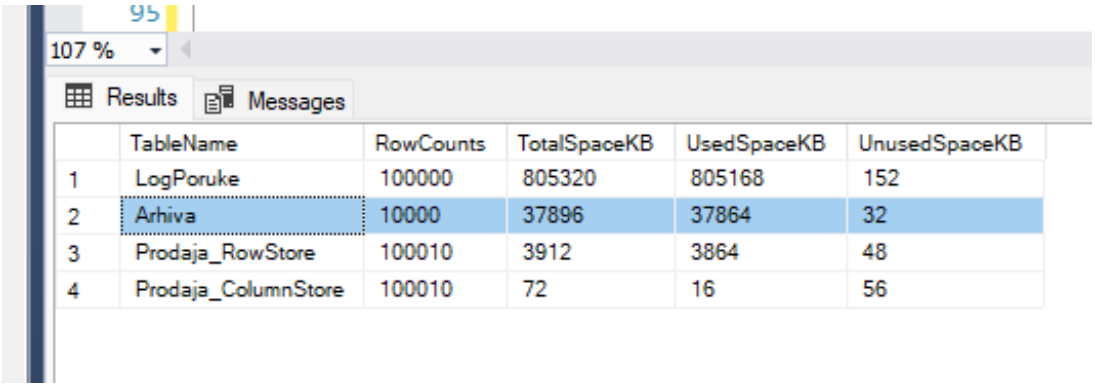
2.4.3 Gde se kompresija fizički primenjuje

- ROW/PAGE kompresija se primenjuje nad row-store strukturama (heap/B-tree) i podešava se po particiji/indeksu
- Kompresija je prvenstveno vezana za in-row podatke; LOB podaci koji su van reda (`varchar(max)`, `nvarchar(max)`, `varbinary(max)`, `xml`) ne dobijaju isti efekat kao klasični in-row podaci

- Kod PAGE kompresije, informacije za prefix/dictionary čuvaju se na samoj stranici

```sql
-- Kreiranje tabele sa PAGE kompresijom
CREATE TABLE Arhiva (
    ArhivaID INT IDENTITY PRIMARY KEY,
    Dokument NVARCHAR(4000),
    DatumKreiranja DATE,
    Kategorija VARCHAR(50)
) WITH (DATA_COMPRESSION = PAGE);

-- Promena nivoa kompresije na postojecoj tabeli
ALTER TABLE Arhiva REBUILD WITH (DATA_COMPRESSION = ROW); -- ili PAGE

-- Procena kompresije
SELECT
    t.NAME AS TableName,
    p.rows AS RowCounts,
    SUM(a.total_pages) * 8 AS TotalSpaceKB,
    SUM(a.used_pages) * 8 AS UsedSpaceKB,
    (SUM(a.total_pages) - SUM(a.used_pages)) * 8 AS UnusedSpaceKB
FROM
    sys.tables t
INNER JOIN
    sys.indexes i ON t.object_id = i.object_id
INNER JOIN
    sys.partitions p ON i.object_id = p.object_id AND i.index_id = p.
   index_id
INNER JOIN
    sys.allocation_units a ON p.partition_id = a.container_id
WHERE
    t.is_ms_shipped = 0
GROUP BY
    t.Name, p.Rows
ORDER BY
    TotalSpaceKB DESC;
```

Listing 5: Implementacija i tačnija procena kompresije podataka



107 %

Results | Messages

	TableName	RowCounts	TotalSpaceKB	UsedSpaceKB	UnusedSpaceKB
1	LogPoruke	100000	805320	805168	152
2	Arhiva	10000	37896	37864	32
3	Prodaja_RowStore	100010	3912	3864	48
4	Prodaja_ColumnStore	100010	72	16	56

Slika 5: Tabela bez kompresije

107 %

Results | Messages

	TableName	RowCounts	TotalSpaceKB	UsedSpaceKB	UnusedSpaceKB
1	LogPoruke	100000	805320	805168	152
2	Arhiva	10000	17456	16480	976
3	Prodaja_RowStore	100010	3912	3864	48
4	Prodaja_ColumnStore	100010	72	16	56

Slika 6: Tabela sa ROW kompresijom

	TableName	RowCounts	TotalSpaceKB	UsedSpaceKB	UnusedSpaceKB
1	LogPoruke	100000	805320	805168	152
2	Prodaja_RowStore	100010	3912	3864	48
3	Arhiva	10000	1008	416	592
4	Prodaja_ColumnStore	100010	72	16	56

Slika 7: Tabela sa PAGE kompresijom

2.4.4 Uticaj kompresije na performanse

Tabela 2: Uticaj kompresije na performanse

Aspekt	**Pozitivan uticaj**	**Negativan uticaj**
I/O operacije	Manje stranica za čitanje	-
Buffer pool	Više podataka u memoriji	-
CPU	-	Overhead za kompresiju/dekompresiju
INSERT/UPDATE	-	Sporije zbog kompresije
SELECT	Brže (manje I/O)	Minimalan CPU overhead

Kompresija je posebno korisna za:

- Tabele sa mnogo ponavljajućih vrednosti
- Istorijske/arhivske podatke koji se retko menjaju
- Sisteme gde je I/O usko grlo, a CPU ima rezerve

2.5 Uticaj organizacije podataka na OLTP vs OLAP sisteme

Izbor fizičke organizacije podataka zavisi od tipa radnog opterećenja [1, 2]:

2.5.1 OLTP (Online Transaction Processing)

OLTP sistemi karakteriše:

- Mnogo kratkih transakcija (INSERT, UPDATE, DELETE)
- Pristup pojedinačnim redovima ili malim skupovima
- Zahtev za niskom latencijom
- Visoka konkurentnost

Preporučena organizacija za OLTP:

- Row-store tabele sa klasterovanim indeksom
- Uski klasterovani ključ (INT IDENTITY)
- Neklasterovani indeksi za česte upite
- ROW kompresija (manji CPU overhead od PAGE)

2.5.2 OLAP (Online Analytical Processing)

OLAP sistemi karakteriše:

- Složeni upiti sa agregacijama
- Čitanje velikih količina podataka
- Manje operacija pisanja (batch load)
- Tolerancija za višu latenciju pojedinačnih upita

Preporučena organizacija za OLAP:

- Columnstore indeksi (klasterovani ili neklasterovani)
- Particionisanje po datumu za efikasno upravljanje
- PAGE kompresija za dodatnu uštedu prostora
- Indeksirani pogledi za česte agregacije

```sql
-- Tabela sa row-store za OLTP i neklasterovanim
-- columnstore za analitiku
CREATE TABLE Transakcije (
    TransakcijaID INT IDENTITY PRIMARY KEY CLUSTERED,
    DatumTransakcije DATETIME2,
    KupacID INT,
    ProizvodID INT,
    Kolicina INT,
    Iznos DECIMAL(12,2),
    -- Neklasterovani columnstore za analiticke upite
    INDEX NCCI_Transakcije_Analitika
        NONCLUSTERED COLUMNSTORE (DatumTransakcije, ProizvodID, Kolicina
    , Iznos)
);
```

Listing 6: Hibridni pristup - HTAP (Hybrid Transactional/Analytical Processing)

Tabela 3: Preporuke za OLTP vs OLAP organizaciju

Aspekt	**OLTP**	**OLAP**
Skladištenje	Row-store	Column-store
Indeksi	B-tree (klasterovani + neklasterovani)	Columnstore
Kompresija	ROW ili bez	PAGE ili Columnstore
Particionisanje	Opciono	Preporučeno (po datumu)
Normalizacija	Visoka (3NF+)	Denormalizovano (star schema)

3 Radno opterećenje (Query Workload)

Polazna tačka fizičkog projektovanja baze podataka je razumevanje radnog opterećenja (workload) - skupa upita i UPDATE operacija koje sistem treba da podrži. Bez poznavanja workload-a, nemoguće je doneti ispravne odluke o indeksima, organizaciji podataka i strategijama optimizacije [1].

3.1 Analiza workload-a po koracima

Analiza workload-a treba da ide sledećim redom [1]:

3.1.1 Najvažniji upiti i koliko se često koriste

Za svaki ključni upit potrebno je odrediti:

- Kojim se relacijama (tabelama) pristupa
- Koji se atributi traže u SELECT iskazu
- Nad kojim atributima se vrši spajanje ili selekcija u WHERE iskazu i koliko su ti uslovi selektivni
- Koliko se često upit izvršava (učestalost) i da li je vremenski kritičan

Primer upita za izdvajanje ključnih SELECT obrazaca i njihove učestalosti:

```sql
-- Najvazniji SELECT upit koji se cesto izvrsava
SELECT TOP (100)
    NarudzbinaID,
    DatumNarudzbine,
    KupacID,
    Iznos
FROM IstorijaNarudzbina
WHERE KupacID = @KupacID
ORDER BY DatumNarudzbine DESC;

-- Ucestalost izvrsavanja po tipu upita
SELECT
    QueryLabel,
    COUNT(*) AS BrojIzvrsavanja,
    AVG(DurationMs) AS ProsecnoTrajanjeMs,
    MAX(DurationMs) AS NajgoriSlucajMs
FROM WorkloadLog
WHERE EventTime >= DATEADD(HOUR, -24, SYSDATETIME())
GROUP BY QueryLabel
ORDER BY BrojIzvrsavanja DESC;
```

Listing 7: Najvazniji upiti i ucestalost (primer)

3.1.2 Najvažnija ažuriranja i koliko se često izvršavaju

Za svaku operaciju ažuriranja potrebno je odrediti:

- Tip ažuriranja (INSERT, DELETE, UPDATE) i relacija koja se ažurira

- Nad kojim atributima se vrši selekcija/spajanje u WHERE/JOIN uslovima i koliko su ti uslovi selektivni
- Za UPDATE komandu - koja polja se menjaju
- Učestalost izvršavanja (npr. broj operacija u minuti/satu)

3.1.3 Željene performanse za upite i ažuriranja

Pre donošenja odluka o indeksima i organizaciji podataka, potrebno je definisati ciljeve performansi:

- Očekivano vreme odziva za najvažnije upite
- Očekivani protok ažuriranja (broj INSERT/UPDATE/DELETE operacija u jedinici vremena)
- Prioritete (koje performanse se moraju garantovati i pod kojim opterećenjem)

Fizičko projektovanje je uvek kompromis: indeks koji ubrzava jedan upit usporava operacije ažuriranja nad istom tabelom. Zato je potrebno razumeti ceo workload pre donošenja odluka [1].

3.2 Preporuke za odluke nakon analize workload-a

Na osnovu analize workload-a, ključne odluke fizičkog projektovanja su:

1. **Koje indekse napraviti** - Koje relacije indeksirati, koja polja izabrati za ključ pretrage, da li indeks treba biti klasterovan ili ne, redak ili gust
2. **Da li menjati konceptualnu shemu** - Razmotriti alternativne normalizovane sheme, denormalizaciju, vertikalno particionisanje ili uvođenje pogleda
3. **Da li preformulisati upite** - Često izvršavani upiti mogu biti preformulisani tako da budu efikasniji za datu konfiguraciju indeksa

3.3 Tuning upita (Query tuning)

Pored podešavanja fizičke strukture podataka i indeksa, značajan deo performansi zavisi i od načina na koji su upiti napisani. U praksi je korisno analizirati plan izvršavanja, `STATISTICS IO` i `STATISTICS TIME`, pa zatim preformulisati upit tako da bude pogodniji za indeksni pristup [5, 2].

3.3.1 SARGable uslovi u WHERE klauzuli

Search ARGument ABLE (SARGable) uslovi su oni koji omogućavaju optimizeru da koristi indekse efikasno.
Funkcija nad kolonom u `WHERE` delu često sprečava efikasan `Index Seek`. Preporuka je da se funkcija pomeri na konstantu/parametar.

```sql
-- Funkcija nad kolonom, cesto vodi ka skeniranju indeksa
SELECT NarudzbinaID, KupacID, DatumNarudzbine
FROM Narudzbine
WHERE YEAR(DatumNarudzbine) = 2025;

-- Bolje resenje: opseg nad kolonom, pogodnije za Index Seek
SELECT NarudzbinaID, KupacID, DatumNarudzbine
FROM Narudzbine
WHERE DatumNarudzbine >= '2025-01-01'
    AND DatumNarudzbine <  '2026-01-01';
```

Listing 8: Pre i posle tuning-a SARGability uslova

3.3.2 Smanjenje čitanja

Upiti koji vraćaju samo potrebne kolone često postižu bolji plan (manje I/O i manji memory grant), posebno kada postoji covering indeks.

```sql
-- Neoptimalan upit: select svih kolona
SELECT *
FROM IstorijaNarudzbina
WHERE KupacID = @KupacID;

-- Optimalniji upit: select samo potrebnih kolona
SELECT NarudzbinaID, DatumNarudzbine, Iznos
FROM IstorijaNarudzbina
WHERE KupacID = @KupacID;
```

Listing 9: Pre i posle tuning-a projekcije kolona

3.3.3 JOIN tuning i rano filtriranje

Kod JOIN upita je poželjno da se selektivni uslovi primene što ranije i da tipovi kolona u spoju budu usklađeni, kako bi se izbegle implicitne konverzije i skupa skeniranja.

```sql
-- Filtriranje nakon join-a i potencijalna implicitna konverzija
SELECT n.NarudzbinaID, k.Ime, k.Prezime
FROM Narudzbine n
JOIN Kupci k ON CONVERT(VARCHAR(20), k.KupacID) = n.KupacIDTekst
WHERE n.Status = 'Aktivna';

-- Poboljsanje: uskladjeni tipovi i direktan JOIN uslov
SELECT n.NarudzbinaID, k.Ime, k.Prezime
FROM Narudzbine n
JOIN Kupci k ON k.KupacID = n.KupacID
WHERE n.Status = 'Aktivna';
```

Listing 10: Primer tuning-a JOIN upita

3.3.4 Poređenje pristupa: JOIN vs CTE vs CROSS APPLY

Za zadatak "poslednja narudžbina po kupcu"često je korisno uporediti više formulacija istog upita, jer optimizer može izabrati različite planove izvršavanja u zavisnosti od oblika upita i dostupnih indeksa.

```sql
-- 1) JOIN + agregacija
SELECT
```

```sql
    k.KupacID,
    n.DatumNarudzbine,
    n.Iznos
FROM Kupci k
LEFT JOIN (
    SELECT KupacID, MAX(DatumNarudzbine) AS MaxDatum
    FROM IstorijaNarudzbina
    GROUP BY KupacID
) x ON x.KupacID = k.KupacID
LEFT JOIN IstorijaNarudzbina n
    ON n.KupacID = x.KupacID
   AND n.DatumNarudzbine = x.MaxDatum;

-- 2) CTE + ROW_NUMBER
WITH PoslednjaNarudzbina AS (
    SELECT
        KupacID,
        DatumNarudzbine,
        Iznos,
        ROW_NUMBER() OVER (
            PARTITION BY KupacID
            ORDER BY DatumNarudzbine DESC, NarudzbinaID DESC
        ) AS rn
    FROM IstorijaNarudzbina
)
SELECT
    k.KupacID,
    p.DatumNarudzbine,
    p.Iznos
FROM Kupci k
LEFT JOIN PoslednjaNarudzbina p
    ON p.KupacID = k.KupacID
   AND p.rn = 1;

-- 3) CROSS APPLY + TOP (1)
SELECT
    k.KupacID,
    p.DatumNarudzbine,
    p.Iznos
FROM Kupci k
OUTER APPLY (
    SELECT TOP (1)
        n.DatumNarudzbine,
        n.Iznos
    FROM IstorijaNarudzbina n
    WHERE n.KupacID = k.KupacID
    ORDER BY n.DatumNarudzbine DESC, n.NarudzbinaID DESC
) p;
```

Listing 11: JOIN, CTE i CROSS APPLY

Uopšteno, `JOIN + agregacija` je često dobar izbor kada se obrađuje veliki deo tabele jer optimizer može efikasno koristiti grupisanje i paralelizaciju. `CTE + ROW_NUMBER()` je najčitljiviji i vrlo stabilan za analitičke obrasce rangiranja, ali može zahtevati više sortiranja/memorije na velikim skupovima. `CROSS APPLY + TOP (1)` je često najbrži za "top-N po grupišcenarije kada postoji odgovarajući indeks (npr. `(KupacID, DatumNarudzbine DESC)`), jer omogućava brze seek operacije po svakoj grupi.

4 Projektovanje tabela i relacija

Projektovanje tabela i relacija predstavlja temelj fizičkog projektovanja baze podataka. Odluke o strukturi tabela, izboru tipova podataka, definisanju ključeva i ograničenja direktno utiču na performanse, integritet podataka i mogućnost kasnijeg skaliranja sistema [1].

4.1 Izbor tipova podataka

Izbor odgovarajućeg tipa podataka za svaku kolonu ima značajan uticaj na performanse jer određuje veličinu reda, efikasnost korišćenja memorije i mogućnost primene indeksa. Osnovni princip je koristiti najuži tip podataka koji može da predstavi sve dozvoljene vrednosti [2, 4].

- **Celobrojni tipovi** - Koristiti `TINYINT` (1 bajt), `SMALLINT` (2 bajta), `INT` (4 bajta) ili `BIGINT` (8 bajtova) u zavisnosti od opsega vrednosti. Na primer, za kolonu koja čuva starost osobe, `TINYINT` (0–255) je dovoljan i koristi 8 puta manje prostora od `BIGINT`
- **Znakovni tipovi** - Koristiti `VARCHAR` umesto `CHAR` za podatke promenljive dužine, jer `CHAR(100)` uvek zauzima 100 bajtova. Koristiti `NVARCHAR` samo kada je potrebna podrška za Unicode znakove, jer zauzima duplo više prostora od `VARCHAR`
- **Datumski tipovi** - Koristiti `DATE` (3 bajta) umesto `DATETIME` (8 bajtova) kada nije potrebna vremenska komponenta. Za vremensku preciznost koristiti `DATETIME2` koji je tačniji i može zauzimati manje prostora od `DATETIME`
- **Decimalni tipovi** - Za novčane vrednosti koristiti `DECIMAL` ili `MONEY` umesto `FLOAT`, jer tipovi sa pokretnim zarezom mogu uzrokovati greške zaokruživanja

```sql
-- LOŠ izbor tipova podataka
CREATE TABLE Zaposleni_Los
(
    ZaposleniID BIGINT IDENTITY,          -- prevelik tip
    Ime NVARCHAR(MAX),                    -- nepotrebno MAX
    Starost FLOAT,                        -- neprecizan za cele brojeve
    Plata FLOAT,                          -- greške zaokruživanja
    DatumZaposlenja DATETIME,             -- nepotrebna preciznost
    Aktivan NVARCHAR(10)                  -- string umesto bita
);

-- DOBAR izbor tipova podataka
CREATE TABLE Zaposleni_Dobar
(
    ZaposleniID INT IDENTITY,             -- dovoljan opseg
    Ime NVARCHAR(100),                    -- ograničena dužina
    Starost TINYINT,                      -- 0-255 je dovoljan
    Plata DECIMAL(12,2),                  -- precizan tip
    DatumZaposlenja DATE,                 -- samo datum
    Aktivan BIT DEFAULT 1                -- logička vrednost
);
```

Listing 12: Izbor tipova podataka – dobar i loš primer

4.2 Projektovanje primarnog ključa

Svaka tabela treba da ima primarni ključ koji jednoznačno identifikuje svaki red. Primarni ključ može biti prirodni (atribut koji ima poslovni značaj) ili surogat (veštački generisan identifikator). Izbor primarnog ključa utiče na performanse jer se on koristi kao klasterovani indeks (podrazumevano u SQL Server-u) i kao referenca u svim neklasterizovanim indeksima [1, 2].

- **Surogat ključevi** - `INT IDENTITY` ili `BIGINT IDENTITY` su najčešći izbor jer su uski (4 ili 8 bajtova), jedinstveni, nepromenjivi i monotono rastući, što minimizira fragmentaciju
- **Prirodni ključevi** - Pogodni kada postoji stabilan, jedinstven poslovni atribut (npr. JMBG, ISBN). Prednost je što eliminišu potrebu za dodatnim indeksom na poslovnom atributu, ali su često širi i podložni promenama
- **Kompozitni ključevi** - Koriste se u asocijativnim tabelama (npr. tabela za vezu M:N). Treba voditi računa o tome da širi ključ povećava veličinu svih neklasterizovanih indeksa koji ga referenciraju

```sql
-- Surogat ključ
CREATE TABLE Kupci
(
    KupacID INT IDENTITY NOT NULL,
    JMBG CHAR(13) NOT NULL,
    Ime NVARCHAR(50) NOT NULL,
    Prezime NVARCHAR(50) NOT NULL,
    Email VARCHAR(100),
    CONSTRAINT PK_Kupci PRIMARY KEY CLUSTERED (KupacID),
    CONSTRAINT UQ_Kupci_JMBG UNIQUE (JMBG)
);

-- Kompozitni ključ u asocijativnoj tabeli
CREATE TABLE KupacProizvod
(
    KupacID INT NOT NULL,
    ProizvodID INT NOT NULL,
    DatumKupovine DATE NOT NULL DEFAULT GETDATE(),
    CONSTRAINT PK_KupacProizvod
        PRIMARY KEY CLUSTERED (KupacID, ProizvodID),
    CONSTRAINT FK_KP_Kupac
        FOREIGN KEY (KupacID) REFERENCES Kupci(KupacID),
    CONSTRAINT FK_KP_Proizvod
        FOREIGN KEY (ProizvodID) REFERENCES Proizvodi(ProizvodID)
);
```

Listing 13: Projektovanje primarnog ključa

Napomena: Detaljnije informacije o fizičkim karakteristikama klasterovanih ključeva (širina, jedinstvenost, statičnost i monotono rastuće vrednosti) i njihovom uticaju na performanse indeksiranja obrađene su u podsekciji 5.2.

4.3 Strani ključevi i referencijalni integritet

Strani ključevi obezbeđuju referencijalni integritet između tabela, garantujući da relacije između podataka ostaju konzistentne. Pored očuvanja integriteta, strani ključevi

pomažu i Query Optimizer-u da generiše efikasnije planove izvršavanja jer pružaju informacije o relacijama između tabela [2].

- **Indeksiranje stranih ključeva** - SQL Server ne kreira automatski indeks na koloni stranog ključa. Bez indeksa, svaka provera referencijalnog integriteta prilikom DELETE ili UPDATE operacije na roditeljskoj tabeli zahteva Table Scan na dete tabeli, što može biti veoma sporo za velike tabele
- **Kaskadne operacije** - `ON DELETE CASCADE` i `ON UPDATE CASCADE` automatski propagiraju promene na povezane redove, ali treba ih koristiti sa oprezom jer mogu uzrokovati neželjeno brisanje velikog broja redova
- **Tip kolone** - Kolona stranog ključa mora biti istog tipa i veličine kao kolona primarnog ključa na koju referira, da bi se izbegla implicitna konverzija koja sprečava korišćenje indeksa

```sql
CREATE TABLE Narudzbine
(
    NarudzbinaID INT IDENTITY NOT NULL,
    KupacID INT NOT NULL,
    DatumNarudzbine DATE NOT NULL DEFAULT GETDATE(),
    Status VARCHAR(20) NOT NULL DEFAULT 'Nova',
    CONSTRAINT PK_Narudzbine
        PRIMARY KEY CLUSTERED (NarudzbinaID),
    CONSTRAINT FK_Narudzbine_Kupac
        FOREIGN KEY (KupacID) REFERENCES Kupci(KupacID)
);

-- SQL Server ne kreira automatski indeks za FK kolonu
CREATE NONCLUSTERED INDEX IX_Narudzbine_Kupac
ON Narudzbine(KupacID);
```

Listing 14: Strani ključevi i indeksiranje

4.4 Ograničenja (Constraints)

Ograničenja obezbeđuju integritet podataka na nivou baze, nezavisno od aplikativnog sloja. Svako ograničenje ujedno pomaže optimizatoru jer sužava skup mogućih vrednosti i omogućava eliminaciju nepotrebnih operacija u planu izvršavanja [1].

- **NOT NULL** - Definisati kolone kao `NOT NULL` gde god je moguće. NULL vrednosti zauzimaju dodatni prostor za NULL bitmap, komplikuju logiku upita i mogu sprečiti korišćenje nekih optimizacija
- **CHECK ograničenja** - Ograničavaju dozvoljene vrednosti kolone. Optimizator može koristiti CHECK ograničenja za eliminaciju nepotrebnih particija ili preskakanje delova podataka
- **DEFAULT vrednosti** - Obezbeđuju da kolone imaju smislenu početnu vrednost, smanjujući pojavu NULL vrednosti i pojednostavljujući INSERT operacije
- **UNIQUE ograničenja** - Garantuju jedinstvenost vrednosti i implicitno kreiraju neklasterovani indeks, koji se može koristiti za brzo pretraživanje

```sql
CREATE TABLE Proizvodi
(
    ProizvodID INT IDENTITY NOT NULL,
    Naziv NVARCHAR(200) NOT NULL,
    Sifra VARCHAR(20) NOT NULL,
    Cena DECIMAL(12,2) NOT NULL,
    KategorijaID INT NULL,
    DatumKreiranja DATETIME2(0) NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT PK_Proizvodi PRIMARY KEY (ProizvodID),
    CONSTRAINT UQ_Proizvodi_Sifra UNIQUE (Sifra),
    CONSTRAINT CK_Proizvodi_Cena CHECK (Cena >= 0),
    CONSTRAINT FK_Proizvodi_Kategorija
        FOREIGN KEY (KategorijaID)
        REFERENCES Kategorije(KategorijaID)
);
```

Listing 15: Definisanje ograničenja

4.5 Particionisanje tabela

Particionisanje tabela je tehnika kojom se velika tabela deli na manje fizičke delove (particije) na osnovu vrednosti jedne kolone, pri čemu tabela logički ostaje jedinstvena. Particije se mogu smestiti na različite filegroup-e i diskove, čime se poboljšavaju performanse i upravljanje podacima [2].

Particionisanje je korisno za:

- Velike tabele (obično preko 10 miliona redova) gde upiti pristupaju samo podskupu podataka
- Tabele sa vremenskim podacima gde se stari podaci arhiviraju ili brišu (sliding window strategija)
- Scenarije gde je potrebno odvojeno održavanje različitih delova podataka (npr. reindeksiranje samo aktivne particije)

4.5.1 HASH particionisanje

SQL Server ne podrzava nativno HASH particionisanje, ali se moze simulirati koriscenjem CHECKSUM funkcije i kalkulativnih kolona. HASH particionisanje omogucava ravnomernu distribuciju podataka izmedju particija.

```sql
-- Dodavanje computed kolone za hash vrednost
ALTER TABLE Narudzbine
ADD PartitionHash AS (ABS(CHECKSUM(KupacID)) % 4) PERSISTED;

-- Partition function na osnovu hash vrednosti
CREATE PARTITION FUNCTION PF_Hash4 (INT)
AS RANGE LEFT FOR VALUES (0, 1, 2, 3);  -- 4 particije: 0,1,2,3

CREATE PARTITION SCHEME PS_Hash4
AS PARTITION PF_Hash4
TO (FG_Hash0, FG_Hash1, FG_Hash2, FG_Hash3);

-- Kreiranje tabele sa hash particionisanjem
CREATE TABLE Narudzbine_Hash (
    NarudzbinaID INT IDENTITY,
```

```sql
    KupacID INT,
    DatumNarudzbine DATE,
    PartitionHash AS (ABS(CHECKSUM(KupacID)) % 4) PERSISTED,
    CONSTRAINT PK_Narudzbine_Hash
        PRIMARY KEY (NarudzbinaID, PartitionHash)
) ON PS_Hash4(PartitionHash);
```

Listing 16: Simulacija HASH particionisanja

Kada koristiti HASH particionisanje:

- Ravnomerna distribucija podataka bez prirodnog opsega
- Load balancing izmedju filegroup-a
- Paralelno procesiranje velikih tabela
- Kada ne postoji prirodni kljuc za RANGE particionisanje

4.5.2 LIST particionisanje

LIST particionisanje deli podatke na osnovu diskretnih vrednosti. U SQL Server-u se implementira kroz RANGE sa eksplicitnim mapiranjem vrednosti na numericke identifikatore.

```sql
-- Mapiranje regiona na numericke vrednosti
-- Srbija=1, Hrvatska=2, BiH=3, Crna Gora=4

-- Partition function
CREATE PARTITION FUNCTION PF_Region (INT)
AS RANGE LEFT FOR VALUES (1, 2, 3);  -- 4 particije

CREATE PARTITION SCHEME PS_Region
AS PARTITION PF_Region
TO (FG_Srbija, FG_Hrvatska, FG_BiH, FG_CrnaGora);

-- Tabela sa LIST particionisanjem
CREATE TABLE Prodaja_Region (
    ProdajaID INT IDENTITY,
    RegionID INT,
    Iznos DECIMAL(10,2),
    DatumProdaje DATE,
    CONSTRAINT PK_Prodaja_Region
        PRIMARY KEY (ProdajaID, RegionID)
) ON PS_Region(RegionID);

-- Upit koji koristi partition elimination
SELECT SUM(Iznos)
FROM Prodaja_Region
WHERE RegionID = 1;  -- Pristupa samo particiji FG_Srbija
```

Listing 17: Simulacija LIST particionisanja po regionima

Kada koristiti LIST particionisanje:

- Podela po geografskim regionima ili drzavama
- Podela po kategorijama proizvoda ili departmanima

- Multi-tenant aplikacije (podela po TenantID)
- Kada svaka particija ima razlicite karakteristike pristupa

4.5.3 Poredjenje strategija particionisanja

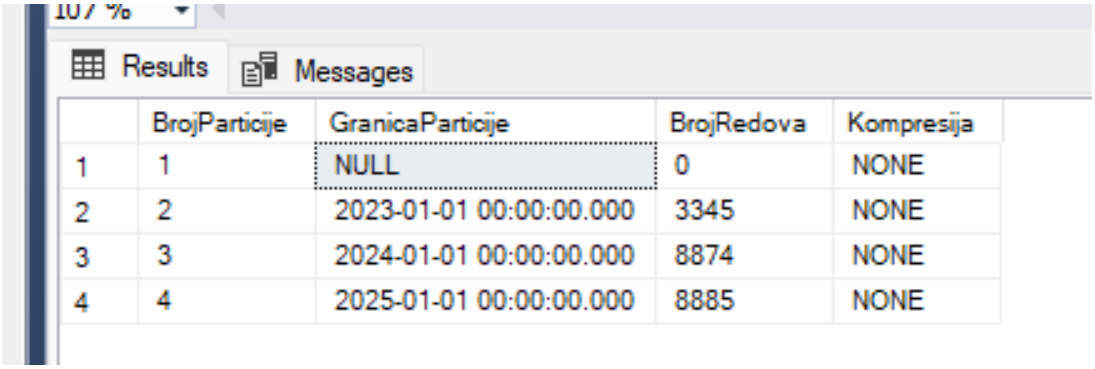
Tabela 4: Poredjenje tipova particionisanja u SQL Server-u

Aspekt	**RANGE**	**HASH**	**LIST**
SQL Server podrska	Ugradjena	Simulacija	Simulacija
Distribucija	Moze biti neravnomerna	Ravnomerna	Zavisi od podataka
Range upiti	Veoma efikasni	Neefikasni	Neefikasni
Point lookup	Efikasan	Veoma efikasan	Veoma efikasan
Tipicna upotreba	Datumi, vremenske serije	Load balancing	Kategorije, regioni
Partition elimination	Da (za range uslove)	Da (za tacnu hash vrednost)	Da (za tacnu vrednost)

```sql
-- 1. Kreiranje particione funkcije
CREATE PARTITION FUNCTION PF_PoGodini (DATE)
AS RANGE RIGHT FOR VALUES
('2023-01-01', '2024-01-01', '2025-01-01');

-- 2. Kreiranje particione sheme
CREATE PARTITION SCHEME PS_PoGodini
AS PARTITION PF_PoGodini
ALL TO ([PRIMARY]);

-- 3. Kreiranje particionisane tabele
CREATE TABLE IstorijaNarudzbina
(
    NarudzbinaID INT IDENTITY NOT NULL,
    DatumNarudzbine DATE NOT NULL,
    KupacID INT NOT NULL,
    Iznos DECIMAL(12,2),
    CONSTRAINT PK_IstorijaNarudzbina
        PRIMARY KEY CLUSTERED (NarudzbinaID, DatumNarudzbine)
) ON PS_PoGodini(DatumNarudzbine);

-- Provera distribucije redova po particijama

SELECT
    p.partition_number          AS BrojParticije,
    prv.value                   AS GranicaParticije,
    p.rows                      AS BrojRedova,
    p.data_compression_desc     AS Kompresija
FROM sys.partitions p
JOIN sys.indexes i
    ON i.object_id = p.object_id AND i.index_id = p.index_id
LEFT JOIN sys.partition_range_values prv
    ON prv.function_id = (
        SELECT pf.function_id
        FROM sys.partition_functions pf
```

```sql
        JOIN sys.partition_schemes ps ON ps.function_id = pf.function_id
        JOIN sys.indexes ix ON ix.data_space_id = ps.data_space_id
        WHERE ix.object_id = p.object_id AND ix.index_id = p.index_id
    )
    AND prv.boundary_id = p.partition_number - 1
WHERE p.object_id = OBJECT_ID('dbo.IstorijaNarudzbina')
  AND i.index_id = 1
ORDER BY p.partition_number;
```

Listing 18: RANGE particionisanje tabele po datumu



Results | Messages

	BrojParticije	GranicaParticije	BrojRedova	Kompresija
1	1	NULL	0	NONE
2	2	2023-01-01 00:00:00.000	3345	NONE
3	3	2024-01-01 00:00:00.000	8874	NONE
4	4	2025-01-01 00:00:00.000	8885	NONE

Slika 8: Raspodela redova po particijama (po godinama)

4.6 Filegroup-e i organizacija skladišta

Filegroup-e omogućavaju logičko grupisanje fajlova baze podataka i kontrolu nad fizičkim raspoređivanjem tabela i indeksa na diskovima. Pravilno korišćenje filegroup-a može poboljšati performanse distribucijom I/O operacija i pojednostaviti administraciju [2].

- **Odvajanje podataka i indeksa** - Smeštanje neklasterovanih indeksa u zasebnu filegroup-u na drugom disku omogućava paralelne I/O operacije prilikom pristupa tabeli i indeksu
- **Filegroup-e za particije** - Svaka particija se može smestiti na zasebnu filegroup-u, što omogućava nezavisno backup-ovanje i restauraciju
- **Read-only filegroup-e** - Istorijski podaci koji se ne menjaju mogu se smestiti u read-only filegroup-u, čime se eliminiše potreba za njihovim backup-ovanjem nakon inicijalnog backup-a

```sql
-- Dodavanje filegroup-a za indekse
ALTER DATABASE ProdajnaBaza ADD FILEGROUP FG_Indeksi;
ALTER DATABASE ProdajnaBaza ADD FILE
(
    NAME = 'ProdajnaBaza_Indeksi',
    FILENAME = 'D:\SQLData\ProdajnaBaza_Indeksi.ndf',
    SIZE = 500MB,
    FILEGROWTH = 100MB
) TO FILEGROUP FG_Indeksi;

-- Kreiranje indeksa na zasebnoj filegroup-i
CREATE NONCLUSTERED INDEX IX_Narudzbine_Datum
```

```
ON Narudzbine(DatumNarudzbine)
ON FG_Indeksi;
```

Listing 19: Organizacija filegroup-a

4.7 Preporuke za projektovanje tabela

Na osnovu analize radnog opterećenja, pri fizičkom projektovanju tabela treba se pridržavati sledećih smernica [1, 2]:

1. **Koristiti najmanje moguće tipove podataka** - Uži redovi pružaju više redova po stranici, manje I/O operacija i efikasnije korišćenje keša
2. **Svaka tabela treba da ima primarni ključ** - Tabela bez primarnog ključa (heap) otežava održavanje i može dovesti do fragmentacije i degradacije performansi
3. **Definisati strane ključeve i indeksirati ih** - Referencijalni integritet treba sprovoditi na nivou baze, a indeksi na FK kolonama sprečavaju skupe Table Scan operacije
4. **Koristiti ograničenja umesto aplikativne logike** - CHECK, DEFAULT i UNIQUE ograničenja su efikasnija od provera u aplikativnom sloju jer se izvršavaju na nivou storage engine-a
5. **Minimizovati upotrebu NULL vrednosti** - NULL vrednosti komplikuju upite, zahtevaju dodatni prostor i mogu sprečiti optimizacije
6. **Particionisati velike tabele** - Za tabele sa više miliona redova gde upiti pristupaju podskupu podataka, particionisanje omogućava partition elimination i brže izvršavanje
7. **Planirati organizaciju filegroup-a unapred** - Distribucija podataka i indeksa na različite diskove poboljšava paralelizam I/O operacija

4.8 Preporuke za podešavanje konceptualne sheme

Ponekad je potrebno izmeniti početnu šemu na osnovu iskustva pri upotrebi baze. Važna napomena je da, pored brige o redundantnosti koja se rešava normalizacijom, šemu treba napraviti i tako da bude u skladu sa upitima i ažuriranjima koja su deo radnog opterećenja [1]. Treba uzeti u obzir sledeće:

- **Izbor normalne forme** - Ponekad je pogodnija 3NF umesto BCNF za dato radno opterećenje, jer BCNF dekompozicija može zahtevati skupa spajanja pri čestim upitima
- **Izbor dekompozicije** - Ukoliko postoje dva načina da se šema dekomponuje, treba izabrati onaj koji je bolji za konkretno radno opterećenje
- **Dodatna dekompozicija** - Ponekad je dobro dalje dekomponovati relaciju koja je već u BCNF, ako to poboljšava performanse za česte upite

- **Denormalizacija** - Ukoliko poboljšava performanse, treba razmisliti o denormalizaciji, odnosno zameni više relacija koje su dobijene normalizacijom jednom većom relacijom, čime se eliminišu skupa spajanja
- **Vertikalno particionisanje** - Razmatranje normalizacije se odnosi na dekompoziciju, kojom bi se dobile dve relacije sa istim šemama, ali sa na primer različitim ograničenjima ili indeksima
- **Horizontalno particionisanje** - Podela relacije na disjunktne podskupove redova (po datumu, regionu, tenant-u i sl.) radi boljeg efekta eliminacije particija, lakšeg održavanja i veće paralelizacije
- **Replikacija** - Razmotriti replikaciju (snapshot, transactional ili merge, prema scenariju) kada je potrebno rasteretiti primarni čvor za čitanja, geografski približiti podatke korisnicima ili povećati dostupnost čitalačkih workload-a

Prilikom menjanja prvobitne šeme treba razmisliti da li je dobro napraviti poglede kojima bi se maskirale ove promene, ukoliko to pojednostavljuje korišćenje baze od strane korisnika kojima je prvobitna šema prirodnija [1].

U praksi, horizontalno particionisanje i replikacija se često kombinuju: particionisanje ubrzava lokalnu obradu i održavanje velikih tabela, dok replikacija omogućava raspodelu čitalačkog opterećenja na sekundarne instance.

5 Projektovanje indeksa

Indeksi predstavljaju najznačajniji mehanizam za optimizaciju performansi upita. Pravilno projektovanje indeksne strategije može poboljšati performanse za redove veličine, dok pogrešno projektovanje može ih pogoršati [5].

5.1 B-tree struktura

SQL Server koristi B+ tree (varijantu B-stabla) kao osnovnu strukturu za indekse [5]. Ključne karakteristike:

- Svi podaci se nalaze na nivou lista
- Non-leaf nivoi služe isključivo za navigaciju
- Leaf stranice su povezane u dvostruko lančanu listu
- Dubina stabla je obično 3-4 nivoa za tabele sa milionima redova
- Pretraga zahteva O(log n) I/O operacija

5.2 Klasterovani indeksi

Klasterovani indeks definiše fizički redosled podataka u tabeli. Leaf nivo klasterizovanog indeksa sadrži kompletne redove tabele. Svaka tabela može imati samo jedan klasterovani indeks [2].

5.2.1 Izbor ključa klasterovanog indeksa

Ključ klasterovanog indeksa treba da bude:

- **Uzak** - Manji ključ = manji neklasterizovani indeksi
- **Jedinstven** - Eliminiše 4-bajtni uniquifier koji SQL Server koristi za duplikate
- **Statičan** - Promena ključa zahteva fizičko premeštanje reda
- **Monotono rastući** - Sprečava podelu stranica i fragmentaciju (IDENTITY constraint)

```sql
-- upotreba IDENTITY ključne reči
CREATE TABLE Narudzbine
(
    NarudzbinaID INT IDENTITY NOT NULL,
    DatumNarudzbine DATE NOT NULL,
    KupacID INT NOT NULL,
    UkupanIznos DECIMAL(18,2),
    CONSTRAINT PK_Narudzbine PRIMARY KEY CLUSTERED (NarudzbinaID)
);

CREATE TABLE Dokumenti
(
    DokumentID UNIQUEIDENTIFIER DEFAULT NEWID() PRIMARY KEY,  -- Nasumič
   ne vrednosti uzrokuju fragmentaciju
```

```
    Naziv VARCHAR(200)
);

-- bolja varijanta: upotreba NEWSEQUENTIALID()
CREATE TABLE Dokumenti
(
    DokumentID UNIQUEIDENTIFIER DEFAULT NEWSEQUENTIALID() PRIMARY KEY,
    Naziv VARCHAR(200)
);
```

Listing 20: Primeri dobrog i lošeg izbora klasterovanog ključa

5.3 Neklasterizovani indeksi

Neklasterizovani indeksi su odvojene B+ tree strukture sa pokazivačima na podatke u osnovnoj tabeli. Na nivou lista sadrže ključ indeksa i row locator (klasterovan ključ ili RID za heap) [5].

5.3.1 Covering indeksi i INCLUDE kolone

Covering indeks sadrži sve kolone potrebne za upit, eliminišući potrebu za Key Lookup operacijom:

```
-- Upit koji zahteva pokrivanje
SELECT KupacID, DatumNarudzbine, Iznos
FROM IstorijaNarudzbina
WHERE KupacID = 361;

-- Covering indeks
CREATE NONCLUSTERED INDEX IX_Narudzbine_Kupac_Cover
ON IstorijaNarudzbina(KupacID)
INCLUDE (DatumNarudzbine, UkupanIznos);
-- INCLUDE kolone se nalaze samo na leaf nivou i ne utiču na sortiranje
   niti veličinu non-leaf nivoa
```

Listing 21: Covering indeks sa INCLUDE kolonama

U tabeli IstorijaNarudzbina je dodato oko 600 000 dummy podataka za različite KupacID vrednosti, a zatim je izvršen upit pre i posle dodavanja covering indeksa radi poređenja performansi:

Tabela 5: STATISTICS IO poređenje pre i posle covering indeksa

Scenario	Scan count	Logical reads	Physical reads	Read-ahead
Pre covering ind.	4	2218	5	2107
Posle covering ind.	4	37	5	29

5.3.2 Kompozitni indeksi

Redosled kolona u kompozitnom indeksu je od ključnog značaja. B-tree sortira podatke po prvoj koloni, zatim po drugoj unutar iste vrednosti prve, itd [5].

```
CREATE NONCLUSTERED INDEX IX_Status_Datum
ON IstorijaNarudzbina(Status, DatumNarudzbine);

-- Index Seek :
```

```sql
SELECT * FROM IstorijaNarudzbina WHERE Status = 'Aktivna';
SELECT * FROM IstorijaNarudzbina
WHERE Status = 'Aktivna' AND DatumNarudzbine > '2024-01-01';

-- indeks se neefikasno koristi (Index Scan ili Table Scan):
SELECT * FROM IstorijaNarudzbina WHERE DatumNarudzbine > '2024-01-01';
```

Listing 22: Uticaj redosleda kolona

5.3.3 Filtrirani indeksi

Filtrirani indeksi indeksiraju samo podskup redova definisan WHERE klauzulom. Manji su, zahtevaju manje prostora i jeftinije ih je održavati [5].

```sql
-- Indeks samo za aktivne narudžbine
CREATE NONCLUSTERED INDEX IX_Narudzbine_Iznos
ON IstorijaNarudzbina(DatumNarudzbine, KupacID)
WHERE Iznos > 5000;

-- Indeks za ne-NULL vrednosti
CREATE NONCLUSTERED INDEX IX_IstorijaNarudzbina_Datum
ON IstorijaNarudzbina(DatumNarudzbine)
WHERE DatumNarudzbine IS NOT NULL;
```

Listing 23: Filtrirani indeksi

5.4 Columnstore indeksi

Columnstore indeksi skladište podatke po kolonama umesto po redovima, što analitičkim upitima omogućava da čitaju samo relevantne kolone umesto celih redova. Kombinacija kolonarnog skladištenja, napredne kompresije i batch mode procesiranja (obrada 900 redova odjednom) čini ih značajno efikasnijim od rowstore indeksa za upite koji agregiraju velike količine podataka. [3].

- **Row Group** - Blok od približno milion redova; osnovna jedinica skladištenja i kompresije
- **Column Segment** - Kompresovani blok podataka za jednu kolonu unutar jedne row grupe
- **Delta Store** - Privremena B-tree struktura koja prima nove redove pre nego što budu kompresovani u row grupu
- **Tuple Mover** - Pozadinski proces koji prebacuje zatvorene Delta Store grupe u kompresovane row grupe
- **Delete Buffer** - Struktura koja prati logički obrisane redove koji još nisu fizički uklonjeni

```sql
-- Klasterovani columnstore indeks za analitičku tabelu
CREATE CLUSTERED COLUMNSTORE INDEX CCI_IstorijaProdaje
ON IstorijaProdaje;

-- Neklasterizovani columnstore indeks za hibridne scenarije
```

```sql
CREATE NONCLUSTERED COLUMNSTORE INDEX NCCI_Prodaja
ON Prodaja(Datum, ProizvodID, Kolicina, Iznos);
```

Listing 24: Columnstore indeksi

5.5 Opšta uputstva za kreiranje indeksa

Prilikom razmatranja koje indekse treba napraviti, kreće se sa listom upita, uključujući i upite koji su deo operacija ažuriranja. Pre nego što se uvede indeks, treba razmisliti kakav uticaj on ima prilikom ažuriranja i u skladu sa tim odlučiti šta je najbolje rešenje. Uputstva za kreiranje indeksa su sledeća [1]:

1. **Da li napraviti indeks** - Ne treba praviti indeks ukoliko on neće doprineti ubrzanju izvršavanja nekog upita, uključujući i one koji su deo ažuriranja. Kada god je moguće, izabrati indeks koji poboljšava više od jednog upita.

2. **Izbor ključa pretrage** - Atributi koji se pominju u WHERE iskazu su kandidati za indeksiranje

3. **Ključ pretrage od više atributa** - Kompozitni indeks treba razmotriti kada:

   - WHERE iskaz sadrži uslov nad više od jednog atributa
   - Indeks omogućava index-only strategiju (pristup relacijama se može izbeći u važnim upitima), čak i sa atributima koji nisu u WHERE iskazu

   Pri kreiranju kompozitnog indeksa, treba biti pažljiv oko redosleda atributa u ključu pretrage.

4. **Klasterovanje** - Najviše jedan indeks nad datom relacijom može biti klasterovan, što veoma povećava performanse. Treba pažljivo odabrati klasterovani indeks:

   - Ukoliko indeks omogućava index-only strategiju za upit koji treba da ubrza, tada nije potrebno da bude klasterovan
   - Klasterovanje ima uticaja samo kod dobavljanja podataka, ne i kod pretrage

5. **Heš indeks naspram B+ stabla** - Uglavnom se preporučuje B+ stablo, zato što je efikasno i za upite intervala i za upite sa tačnom vrednošću. Heš indeks je bolji u situaciji kada postoje veoma važni upiti jednakosti, a pri tome ne postoje upiti intervala nad atributima koji čine ključ pretrage

6. **Balansiranje cene ažuriranja indeksa** - Nakon sastavljanja liste željenih indeksa, treba razmotriti njihov uticaj na ažuriranja u radnom opterećenju:

   - Ukoliko održavanje indeksa usporava operaciju ažuriranja, razmotriti njegovo odbacivanje
   - Treba imati na umu da dodavanje indeksa može ubrzati i same operacije ažuriranja (npr. brže pronalaženje redova za UPDATE)

6 Ko-klasterovanje (Co-clustering)

Ko-klasterovanje (co-clustering) je tehnika fizičkog projektovanja koja podrazumeva skladištenje povezanih podataka iz različitih tabela fizički blizu na disku, čime se smanjuje broj I/O operacija prilikom izvršavanja upita sa JOIN operacijama [1].

6.1 Princip ko-klasterovanja

Osnovna ideja je da se redovi iz dve tabele koji se često zajedno pristupaju povežu na istim stranicama ili u istim opsezima na disku. Na taj način, kada se pristupi redu iz jedne tabele, povezani redovi iz druge tabele su već u memoriji ili na susednim stranicama.

Na primer, ako se tabele **Narudzbine** i **StavkeNarudzbine** često pridružuju po **NarudzbinaID**, ko-klasterovanje bi podrazumevalo da se stavke za određenu narudžbinu skladište fizički blizu te narudžbine.

6.2 Ko-klasterovanje u MS SQL Server-u

SQL Server ne podržava eksplicitno povezivanje redova iz različitih tabela na istim stranicama. Međutim, postoje tehnike koje postižu sličan efekat [1, 2]:

- **Zajednička filegroup strategija** - Smeštanje povezanih tabela u istu filegroup na istom disku obezbeđuje fizičku bliskost podataka
- **Klasterizovani indeks po stranom ključu** - Ako se tabela **StavkeNarudzbine** klasterizuje po **NarudzbinaID**, sve stavke za istu narudžbinu biće fizički zajedno, što ubrzava JOIN operacije
- **Indeksirani pogledi** - Materijalizacija rezultata JOIN-a eliminiše potrebu za pristupom dvema tabelama u realnom vremenu

```sql
-- Klasterovanje podređene tabele po stranom ključu
CREATE TABLE StavkeNarudzbine
(
    StavkaID INT IDENTITY,
    NarudzbinaID INT NOT NULL,
    ProizvodID INT NOT NULL,
    Kolicina INT,
    Cena DECIMAL(18,2),
    CONSTRAINT PK_Stavke
        PRIMARY KEY NONCLUSTERED (StavkaID)
);

-- Klasterovani indeks po stranom ključu
-- za ko-klasterovanje sa tabelom Narudzbine
CREATE CLUSTERED INDEX IX_Stavke_Narudzbina
ON StavkeNarudzbine(NarudzbinaID);
```

Listing 25: Simulacija ko-klasterovanja

Ovaj pristup je najkorisniji za scenarije gde se pretežno čita po relaciji roditelj-dete (npr. narudžbina sa svim stavkama), jer redovi dete tabele za istog roditelja leže na istim ili susednim stranicama [1].

6.3 Preporuke za primenu ko-klasterovanja

Sumarizovano, ko-klasterovanje ima sledeće karakteristike [1]:

- **Prednosti:**
  - Može značajno ubrzati spajanje (JOIN), posebno spajanje po stranom ključu koje odgovara relacijama sa kardinalnošću 1:N
  - Smanjuje broj I/O operacija jer se povezani podaci nalaze na istim ili susednim stranicama
- **Nedostaci:**
  - Sekvencijalna pretraga bilo koje od relacija postaje sporija, jer su redovi jedne relacije raspršeni među redovima druge
  - Ažuriranje postaje sporije zbog potrebe za održavanjem fizičke organizacije podataka

Ko-klasterovanje je posebno korisno u slučajevima kada postoji više nivoa hijerarhije (npr. narudžbina → stavke → detalji) i kada se podaci pretežno čitaju po tim hijerarhijskim vezama.

7 Zaključak

Razumevanje radnog opterećenja (workload) predstavlja polaznu tačku svakog fizičkog projektovanja. Bez poznavanja najvažnijih upita, njihove učestalosti i zahteva za performansama, nemoguće je doneti ispravne odluke o indeksima i organizaciji podataka.

Projektovanje tabela i relacija čini temelj celokupnog fizičkog projektovanja. Izbor odgovarajućih tipova podataka, definisanje primarnih i stranih ključeva, ograničenja i particionisanje tabela direktno utiču na efikasnost skladištenja i brzinu pristupa podacima.

Projektovanje indeksa predstavlja najznačajniji aspekt optimizacije performansi. Pravilna strategija indeksiranja - od izbora ključa klasterovanog indeksa, preko covering indeksa i filtered indeksa, do columnstore indeksa za analitičke scenarije - može znatno poboljšati performanse upita.

Ko-klasterovanje omogućava smanjenje I/O operacija za upite koji pristupaju povezanim tabelama.

8 Literatura

[1] Raghu Ramakrishnan i Johannes Gehrke. *Database Management Systems*. 3rd. New York, NY: McGraw-Hill, 2003. ISBN: 978-0072465631.

[2] Randolph West i dr. *SQL Server 2022 Administration Inside Out*. Redmond, WA: Microsoft Press, 2023. ISBN: 978-0137899883.

[3] Per-Åke Larson i dr. „Enhancements to SQL Server Column Stores". U: *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*. New York, NY, USA: ACM, 2013., str. 1159–1168. DOI: 10.1145/2463676.2463708.

[4] Microsoft Corporation. *SQL Server 2025 Technical Documentation*. Microsoft. 2025. URL: https://learn.microsoft.com/en-us/sql/sql-server/ (posećeno 10. 12. 2025.).

[5] Grant Fritchey. *SQL Server 2022 Query Performance Tuning: Troubleshoot and Optimize Query Performance*. 6th. New York, NY: Apress, 2022. ISBN: 978-1484288900.